Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251188**
Nama Lengkap	**ANGELA REVALINE SETIAWAN**
Minggu ke / Materi	**07 / Percabangan dan Perulangan Kompleks**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Link Github : https://github.com/angelarevalines/71251188_valine.git

1. Struktur Percabangan Kompleks

Struktur percabangan kompleks berarti percabangan di mana kondisi pemilihan tidak hanya satu tetapi bisa terdiri atas banyak alternatif. Perintah-perintah yang dikerjakan juga bisa lebih dari satu dan kompleks.

Contoh percabangan kompleks bentuk 1 :

```
if kondisi1:
     if kondisi2:
```

Contoh percabangan kompleks bentuk 2 :

```
if kondisi1:
     if kondisi2:
     else:
else:
```

Contoh percabangan kompleks bentuk 3 :

```
if kondisi1:
     if kondisi2:
else:
     if kondisi3:
     else:
```

Contoh percabangan kompleks bentuk 4 :

```
if kondisi1:
     if kondisi2:
     else:
else:
     if kondisi3:
     else:
```

Contoh percabangan kompleks bentuk 5 :

```
if kondisi1:
     if kondisi2:
          if kondisi3:
               if kondisi4:
```

Contoh percabangan kompleks bentuk 6:

```
if kondisi1:
else:
     if kondisi2:
     else:
          if kondisi3:
          else:
               if kondisi4:
               else:
```

IF bertingkat memiliki kelebihannya tersendiri. Dengan menggunakan IF bertingkat, maka eksekusi perintah dapat menjadi lebih baik dan efisien, karena tidak semua kondisi IF dikerjakan. Sedangkan pada IF biasa, semua kondisi IF pasti akan dicoba satu per satu walau mungkin pada akhirnya hanya satu saja perintah IF yang terpenuhi. Tentunya hal ini akan memperlambat proses eksekusi.

Perbedaan IF bertingkat dan IF biasa :

1) IF Bertingkat

```
if kondisi1:
     instruksi1
elif kondisi2:
     instruksi2
elif kondisi3:
     instruksi3
elif kondisi4:
     instruksi4
```

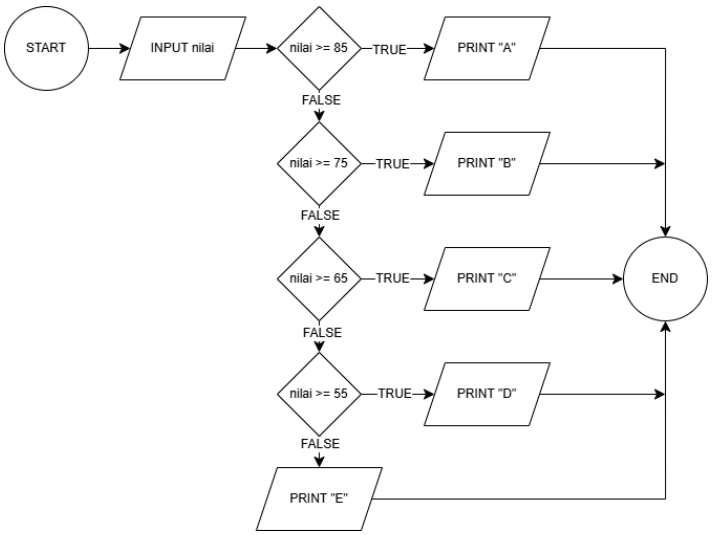
2) IF Biasa

```
if kondisi1:
     instruksi2
if kondisi1:
     instruksi2
if kondisi1:
     instruksi2
if kondisi1:
     instruksi2
```

Contoh kasus yang cocok untuk IF bertingkat adalah konversi nilai angka menjadi nilai huruf dengan batasan-batasan nilai yang sudah ditentukan sebelumnya.

```
nilai = int(input("Masukkan nilai : "))
if nilai >= 85:
     print("Predikat = A")
elif nilai >= 75:
     print("Predikat = B")
elif nilai >= 65:
     print("Predikat = C")
elif nilai >= 55:
     print("Predikat = D")
else:
     print("Predikat = E")
```

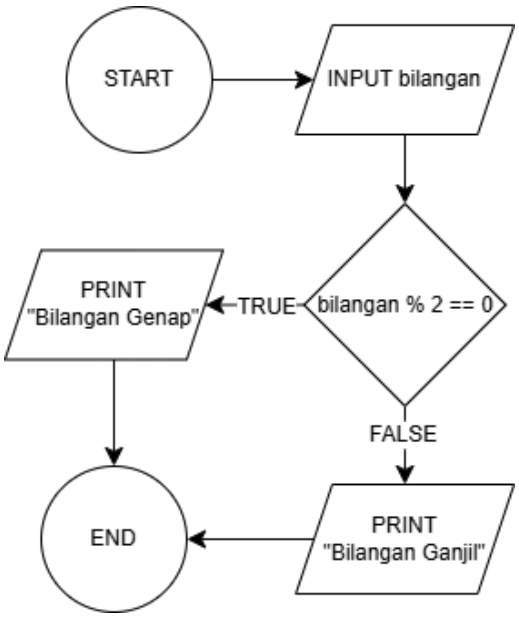
Berikut adalah bentuk flowchartnya :



Contoh kasus untuk IF biasa :

```
bilangan = int(input("Masukkan bilangan : "))
if bilangan % 2 == 0:
     print("Bilangan Genap")
else:
     print("Bilangan Ganjil")
```

Berikut adalah bentuk flowchartnya :



2. Struktur Perulangan Kompleks

A) Break

Perintah ini biasa digunakan untuk menghentikan proses perulangan yang sedang terjadi. Biasanya disebabkan oleh suatu kondisi tertentu yang diimplementasikan menggunakan perintah IF.

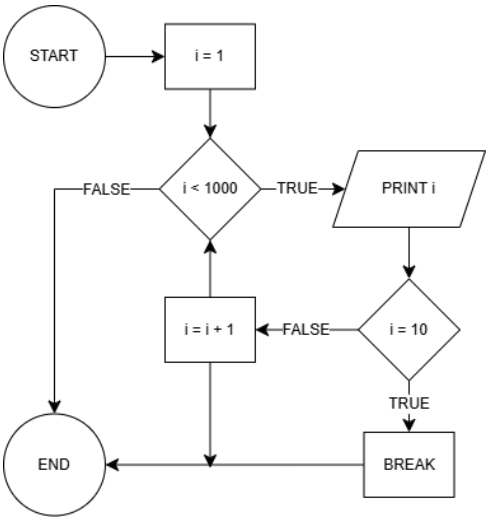
Contoh program **break** dan outputnya :

```
for i in range(1000):
    if i == 10:
        break
    print(i)
```

```
0
1
2
3
4
5
6
7
8
9
```

Program diatas hanya akan menampilkan output angka 1 sampai 9 walaupun di perulangan sudah diset dari angka 1 sampai 1000. Hal ini terjadi karena perintah break yang diberikan pada saat kondisi i = 10. Angka 10 sendiri tidak ikut ditampilkan karena perintah untuk mencetak diletakkan setelah perintah break.

Bentuk flowchart :



B) Continue

Perintah continue menyebabkan proses perulangan kembali ke awal mula, dengan mengabaikan statement-statement berikutnya setelah continue. Biasanya perintah ini juga diimplementasikan menggunakan perintah IF.

Contoh program **continue** dan outputnya :

```
for i in range(10):
    if i == 5:
        continue
    print(i)
```

```
0
1
2
3
4
6
7
8
9
```

Program diatas tidak menampilkan angka 5 karena pada saat angka 5 akan ditampilkan, terdapat perintah continue. Sehingga perintah mencetak di bagian bawahnya tidak akan dikerjakan dan akan langsung dilanjutkan ke perulangan berikutnya.

C) Perulangan Bertingkat

```
for i in range (9):
    for j in range(5):
        print(f"i = {i}, j = {j}")
```

Sumber Materi : Modul 07 Percabangan dan Perulangan Kompleks (LMS)

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Link Github : https://github.com/angelarevalines/71251188_valine.git

SOAL 1

Source Code :

```
def cekangkaprima(angka):
    if angka <= 1:
        return False
    for i in range(2, int(angka**0.5) + 1):
        if angka % i == 0:
            return False
    return True

def angkaprimaterdekat(n):
    for i in range(n - 1, 1, -1):
        if cekangkaprima(i):
            return i

n = int(input("Masukkan bilangan : "))
if n <= 1:
    print("angka tidak valid")
else:
    prima = angkaprimaterdekat(n)
print(f"Bilangan prima terdekat kurang dari {n} adalah {prima}")
```

Penjelasan :

1. Membuat fungsi bernama **cekangkaprima** yang berisi parameter angka untuk mengecek apakah suatu bilangan adalah prima.

2. Ketika angka <= 1 maka program mengembalikan nilai False (bukan prima).
3. Perulangan mulai dari 2 sampai akar dari angka. Jika angka habis dibagi oleh bilangan lain, maka program mengembalikan nilai False. Jika tidak ada pembagi lagi, maka program mengembalikan nilai True (prima).
4. Membuat fungsi baru bernama **angkaprimaterdekat** yang berisi parameter n untuk mencari bilangan prima terbesar yang kurang dari n.
5. Perulangan mulai dari n-1 hingga 2 (mundur).
6. Menggunakan fungsi input untuk pengguna memasukkan bilangan yang ingin di cek yang disimpan dalam variabel **n**.
7. Ketika n <= 1 maka program akan mengeluarkan output "tidak valid" karena tidak ada bilangan prima yang kurang dari 2. Tetapi jika n > 1 maka program akan mengecek dan mengeluarkan output bilangan prima yang terdekat dan kurang dari n.

Output :

```
Masukkan bilangan : 21
Bilangan prima terdekat kurang dari 21 adalah 19
```

SOAL 2

Source Code :

```
def deret(angka):
    for i in range(angka, 0, -1):
        faktorial = 1
        for j in range(1, i + 1):
            faktorial *= j

        print(faktorial, end=' ')
        for k in range(i, 0, -1):
            print(k, end=' ')
        print()

angka = int(input("Masukkan nilai n = "))
if angka < 1:
    print("Nilai n harus lebih besar dari 1")
else:
```

```
    deret(angka)
```

Penjelasan :

1. Membuat fungsi bernama deret yang berisi parameter angka yang berfungsi untuk menampilkan faktorial dan urutan bilangan menurun untuk setiap i dari angka ke 1.
2. Perulangan i mengeksekusi dari angka ke 1.
3. Menggunakan fungsi input agar pengguna dapat memasukkan nilai angka sesuai dengan keinginannya.
4. Tetapi jika angka yang diinput < 1 maka program akan mengeluarkan output “Nilai n harus lebih besar dari 1”, karena jika kurang dari 1 tidak akan mendapat hasil faktorialnya.
5. Jika angka yang diinput sudah valid, maka program akan mengeksekusi deret faktorialnya dan bilangan menurun.

Output :

```
Masukkan nilai n = 6
720 6 5 4 3 2 1
120 5 4 3 2 1
24 4 3 2 1
6 3 2 1
2 2 1
1 1
```

SOAL 3

Source Code :

```
tinggi = int(input("tinggi = "))
lebar = int(input("lebar = "))


angka = 1
for i in range(tinggi):
    for j in range(lebar):
        print(angka, end=' ')
        angka += 1
    print()
```

Penjelasan :

1. Membuat variabel bernama tinggi dan lebar yang menggunakan fungsi input agar pengguna dapat memasukkan nilai tinggi dan lebar yang mereka inginkan.
2. Set angka = 1, berarti berapapun input yang diberikan pengguna, program akan mengeluarkan output dimulai dari 1.
3. Menggunakan fungsi perulangan (for) i yang membaca variabel tinggi yang didalamnya terdapat fungsi perulangan (for) j yang membaca variabel lebar.
4. Jika lebar sudah didalam perulangan tinggi, maka program akan menampilkan angka dengan formasi yang ada di dalam perulangan, dengan menggunakan tambahan end="" supaya outputnya print ke samping.
5. Setelah program menampilkan angka, maka angka otomatis bertambah 1 (angka += 1) dan akan kembali ke perulangan hingga batas yang diinputkan pengguna.

Output :

```
tinggi = 5
lebar = 4
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
17 18 19 20
```